Chapter – 1

INTRODUCTION

1.1 INTRODUCTION OF THE SYSTEM

The Business Website for MT Kinetics is a fully functional, web-based platform designed to digitalize and streamline the operations of MT Kinetics—a trekking and eco-tourism agency based in Mangalore, Karnataka. The system is tailored to address the increasing demand for accessible, user-friendly, and professional online services in the adventure travel industry.

This web application serves as a digital storefront and booking portal, enabling users (primarily adventure enthusiasts and nature lovers) to browse trekking packages, explore detailed itineraries, view customer reviews, and make secure online bookings. From the client's perspective, it simplifies tour management by offering a centralized admin dashboard to manage trek schedules, user bookings, and payment records.

The application ensures end-to-end automation of the booking process, starting from displaying trek packages to payment processing and confirmation. It integrates seamlessly with third-party services such as payment gateways for secure transactions and can be deployed on a cloud-based server for scalability.

1.1.1 Project Title

Tour package Booking System for Mt Kinetics

1.1.2. Category

Project category- Web Application

1.1.3. Overview

MT Kinetics is a system that is designed to modernize and automate the traditional tour booking process by integrating several core functionalities into a single platform. It provides detailed trek information, visual galleries, pricing, itineraries, and user reviews—all aimed at helping customers make informed booking decisions. The booking workflow is tightly integrated with secure payment gateways enabling seamless and protected online transactions.

The backend system allows administrators to manage treks, bookings, customer data, and support queries through a centralized admin dashboard. This not only enhances operational efficiency but also reduces the administrative workload of handling bookings manually. Additionally, the system supports live chat and contact forms to ensure responsive customer service.

1.2 BACKGROUND

MT Kinetics is a company that has built a strong reputation for offering safe, guided trekking experiences that promote nature appreciation, physical well-being, and sustainable tourism practices. The motivation behind the project is to digitally transform the company's operations, improve accessibility for customers, and streamline administrative workflows. By integrating core functionalities such as online trek browsing, secure payment processing, real-time booking management, and customer support, the platform offers a modern solution that aligns with current digital trends in tourism.

1.3 OBJECTIVES OF THE SYSTEM

- To Enable users to explore trek packages online
- To Provide user registration and login functionalities
- To Allow customers to submit booking requests without payment
- To Display detailed trek information, including photos and itineraries
- To Allow administrators to manage trek listings dynamically
- To Provide an admin dashboard to view and approve bookings

1.4 IMPORTANCE AND SCOPE OF THE SYSTEM

MT Kinetics Business Website serves as a strategic response to the rising demand for digitized experiences in the adventure tourism sector. By shifting from manual, fragmented booking methods to a unified, automated web-based system, the platform ensures consistent, real-time engagement between customers and the business. It empowers MT Kinetics to operate with greater precision, scalability, and agility—enhancing customer satisfaction while minimizing administrative overhead.

The system plays a vital role in streamlining core operations such as trek promotion, booking management, and payment reconciliation, all while delivering a responsive and accessible user interface tailored to the expectations of modern consumers.

1.5 STRUCTURE OF THE SYSTEM

Structure is the type of connection between the elements of a whole. It has its own internal dialectic. Wholeness must be composed in a certain way; its parts are always related to the whole. A system consists of something more than structure: it is a structure with certain properties.

The MT Kinetics Trek Management System is built on a layered, modular architecture that separates responsibilities into logical components, ensuring both scalability and maintainability. The system is structured around three primary layers: the Presentation Layer, the Application Layer, and the Data Layer, each with its own clearly defined role in handling user interaction, business logic, and data management.

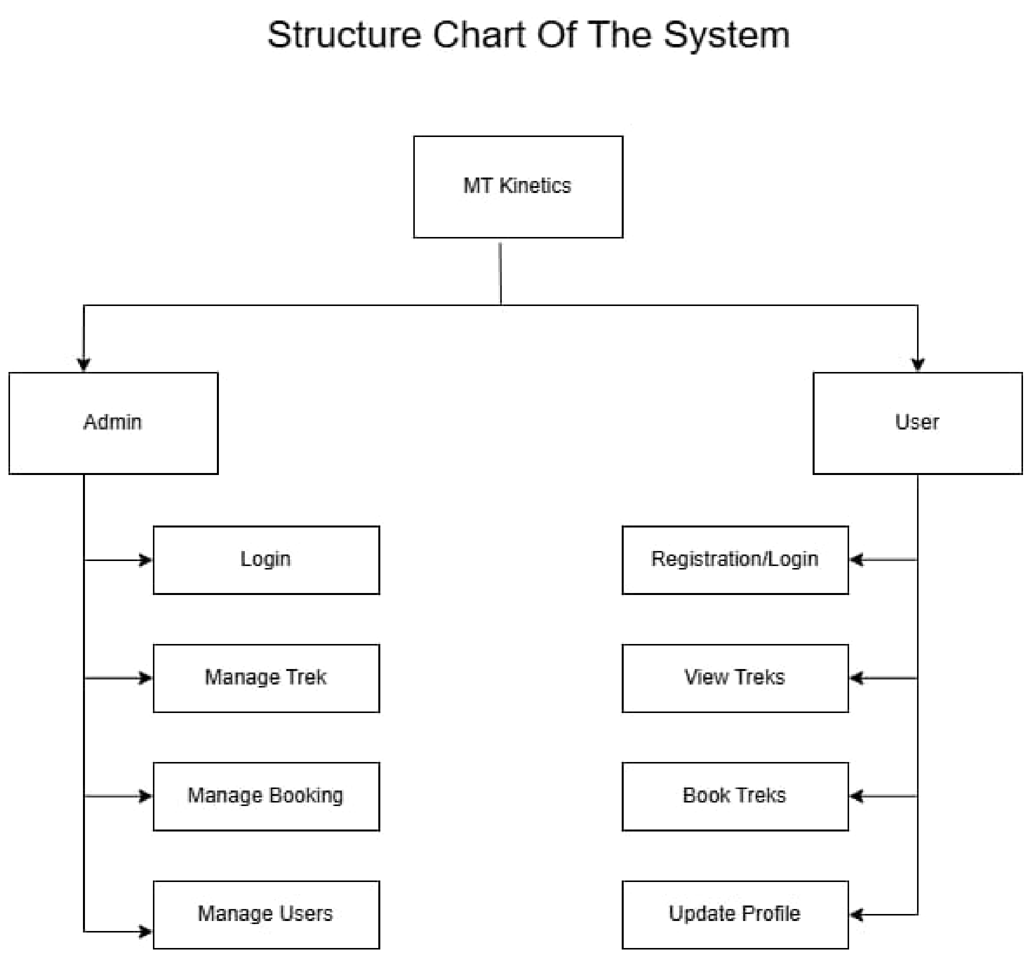


Figure 1.1: Structure of the system

The Application Layer, powered by PHP, acts as the core logic centre of the system. It handles user authentication, input validation, trek and booking management, and communication between the frontend and the database. This layer is also responsible for enforcing business rules, such as booking availability and user access control.

At the foundation, the Data Layer uses MySQL to persistently store information such as user accounts, trek details, and booking records. The database is structured using relational tables with foreign key relationships to ensure data consistency and integrity. All queries and data transactions are securely managed through the PHP backend.

Together, these layers form a robust, cleanly organized system that supports efficient user interaction, secure data handling, and future system expansion without disrupting existing functionality.

1.6 SYSTEM ARCHITECTURE

The system architecture basically shows the fundamental structures of a software system and discipline of creating such structures and systems. Each structure comprises software elements, relations among them and properties of both elements and relations.

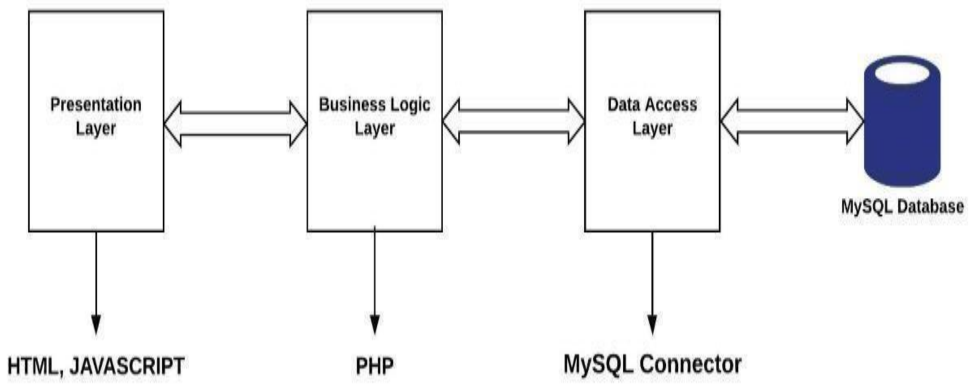


Figure 1.2: System Technical Architecture

Presentation Layer (Frontend / Client-Side)

This layer is responsible for rendering the user interface, displaying information to the user, capturing user input, and managing client-side interactivity to provide a seamless and responsive user experience.

Business Logic Layer (Backend / Application Server - PHP)

This layer is responsible for executing the core business logic and rules of the MT Kinetics application. It processes all dynamic requests, manages complex operations such as booking validation, user authentication, and data manipulation, and orchestrates the interaction between the Presentation Layer and the Data Layer.

Data Access Layer (MYSQL)

This layer acts as the primary gateway for all incoming client requests, serving as the hosting environment for the core application logic. It efficiently manages web traffic, distributes requests to the appropriate application components, and provides the necessary infrastructure for the backend services to operate and connect to the database.

MySQL Database

This is the persistent data store of the entire system. MySQL, being a Relational Database Management System (RDBMS), organizes data into structured tables with predefined schemas.

1.7 SOFTWARE USED FOR DEVELOPMENT

- Operating System: Windows 11
- Web Server: XAMPP
- Programming Languages: PHP, CSS, HTML, Java Script
- Web browsers: Google chrome/Microsoft Edge
- Database: MYSQL
- Text Editor: Microsoft VS Code

1.8 SOFTWARE REQUIRED FOR IMPLEMENTATION

- Processor :1 GHZ or higher CPU

- Hard disk: 500 MB available internal storage
- Memory :512 MB of RAM is minimally recommended
- Display: 2.8 inches or larger

WINDOWS 11

Windows 11 is the latest operating system developed by Microsoft, succeeding Windows 10. It brings a fresh and modern user experience with a redesigned interface, aiming to enhance productivity, creativity, and ease of use. The Start menu is centred and simplified, task management is more intuitive, and multitasking is improved with new features like Snap Layouts and Virtual Desktops. Windows 11 also focuses heavily on integration—bringing Microsoft Teams into the taskbar for quick communication and offering better connectivity between devices and services. With enhanced support for gaming, touch, and pen input, Windows 11 is designed for a wide range of users, including professionals, gamers, and casual users. In essence, Windows 11 represents a significant step forward in user interface design and workflow efficiency, while continuing to build on the stability and compatibility of previous Windows versions.

PHP Language

PHP is a popular general-purpose scripting language that is especially suited to web development. It was created by Rasmus Lerdorf in 1994; the PHP reference implementation is now produced by The PHP Group. PHP originally stood for Personal Home Page, but it now stands for the recursive initialize PHP: Hypertext Preprocessor.

PHP code is usually processed on a web server by a PHP interpreter implemented as a module, a daemon or as a Common Gateway Interface (CGI) executable. On a web server, the result of the interpreted and executed PHP code – which may be any type of data, such as generated HTML or binary image data – would form the whole or part of an HTTP response. Various web template systems, web content management systems, and web frameworks exist which can be employed to orchestrate or facilitate the generation of that response. Additionally, PHP can be used for many programming tasks outside of the web context, such as standalone graphical applications and robotic drone control. Arbitrary PHP code can also be interpreted and executed via the command line interface (CLI).

The standard PHP interpreter, powered by the Zend Engine, is free software released under the PHP License. PHP has been widely ported and can be deployed on most web servers on almost every operating system and platform, free of charge.

Benefits of PHP

- PHP runs on various platforms (Windows, Linux, UNIX, Mac OS X, etc.)
- PHP is compatible with almost all servers used today (Apache, IIS, etc.)
- PHP supports a wide range of databases
- PHP is free.

Apache Web-Server

Apache is an open-source and free web server software that powers around 40% of websites around the world. The official name is Apache HTTP Server, and it's maintained and developed by the Apache Software Foundation. It allows website owners to serve content on the web — hence the name "web server." It's one of the oldest and most reliable web servers, with the first version released more than 20 years ago, in 1995.When someone wants to visit a website, they enter a domain name into the address bar of their browser. Then, the web server delivers the requested files by acting as a virtual delivery man. Here at Hosting, our web hosting infrastructure uses Apache in parallel with NGINX, which is popular web server software. This particular setup allows us to get the best of both worlds. It greatly improves server performance by compensating the weaker sides of one software with the strengths of another.

MySQL Database

MySQL is an open-source relational database management system (RDBMS). Its name is a combination of "My", the name of co-founder Michael Wideness's daughter, and "SQL", the abbreviation for Structured Query Language. MySQL is free and open-source software under the terms of the GNU General Public License and is also available under a variety of proprietary licenses. MySQL was owned and sponsored by the Swedish company MySQL AB, which was bought by Sun Microsystems (now Oracle Corporation). In 2010, when Oracle acquired Sun, Wideness forked the open-source MySQL project to create MariaDB. MySQL is a component of the LAMP web application software stack (and others), which is an acronym for Linux, Apache, MySQL, Perl/PHP/Python. MySQL is used by many database-driven web applications, including Drupal, Joomla, and Word Press. MySQL is also used by many popular websites, including Facebook, Flickr, Media Wiki, Twitter, and YouTube.

Text Editor

Visual Studio Code (VS Code) is a powerful, open-source code editor developed by Microsoft, known for its speed, flexibility, and extensive feature set. Although lightweight in size, VS Code provides the core functionalities of a full Integrated Development Environment (IDE), making it a popular choice among developers for a wide range of programming tasks. VS Code supports multiple programming languages out of the box, including HTML, CSS, JavaScript, PHP, Python, C++, and many more. Its intelligent features like syntax highlighting, IntelliSense (smart code completion), error detection, and code navigation help developers write cleaner and more efficient code.

One of the key strengths of VS Code lies in its extensibility. This includes support for additional programming languages, themes, debuggers, and tools like Docker, Git, and Jupiter Notebooks. VS Code also includes a built-in terminal, allowing developers to run system commands or scripts directly within the editor.

Chapter –2
METHODOLOGY

2.1 SYSTEM DESIGN

System design translates project requirements into a well-defined architecture. It outlines the system's components, their functionalities, and how they interact. It considers scalability, performance, security, and maintainability to ensure a robust and efficient solution. This blueprint guides development and ensures all parts work together seamlessly to achieve the project's goals.

2.1.1 Functional Decomposition

The functional decomposition is term which is used to describe a set of steps in which they break down the overall function of a device, system or process into its smaller parts. Functional decomposition refers broadly to the process of resolving a functional relationship into its constituent parts in such a way that the original can be reconstructed from those parts by function components.

a. System Software Architecture

Software architecture is, simply, the organization of a system. This organization includes all components, how they interact with each other, the environment in which they operate, and the principles used to design the software. In many cases, it can also include the 3 evolutions of the software into the future. Software architecture is designed with a specific mission or missions in mind. That mission has to be accomplished without hindering the missions of other tools or devices. The behaviour– and structure of the software impact significant decisions, so they need to be appropriately rendered and built for the best possible results.

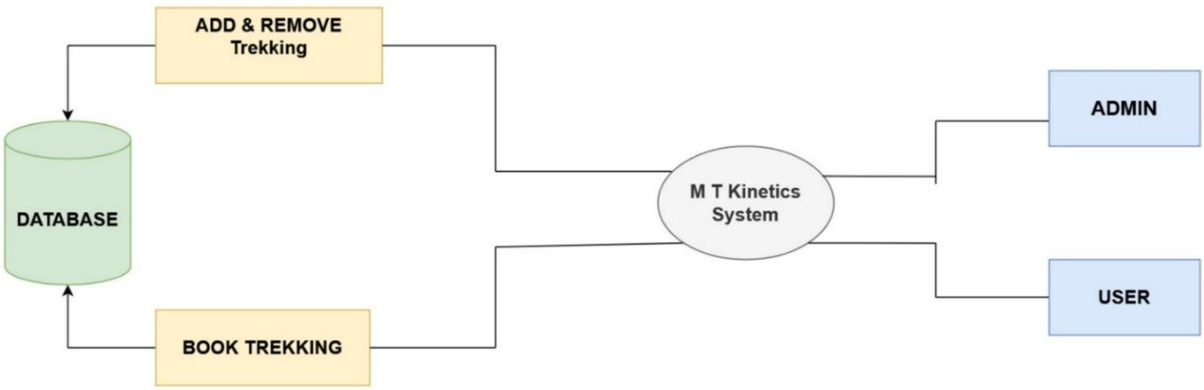


Figure 2.1 System Software Architecture

b. System Technical Architecture

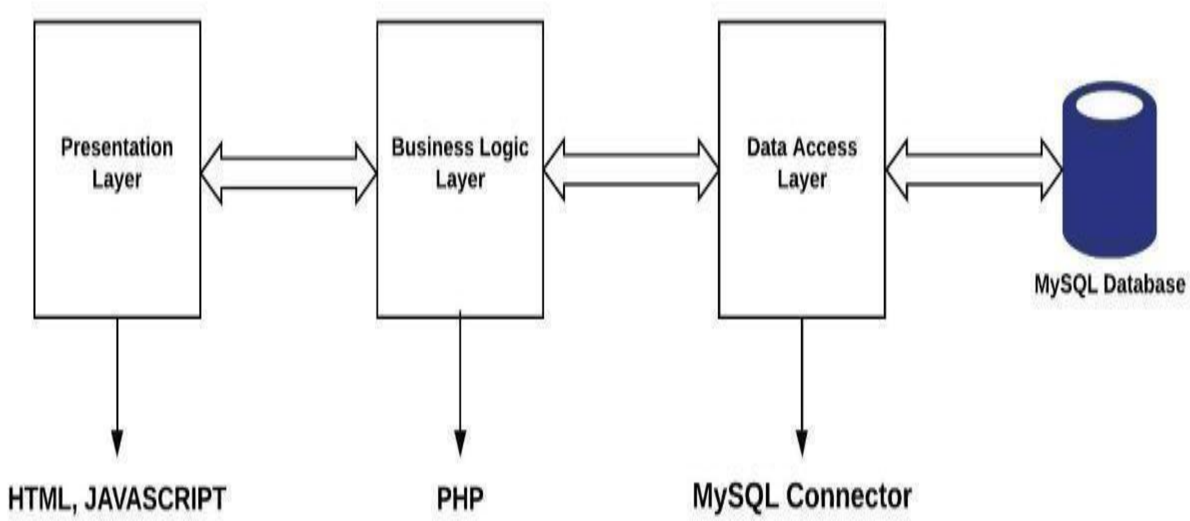


Figure 2.2: System Technical Architecture

The system is made up of four layers:

The presentation layer: includes user interfaces built using the HTML5 and CSS tools. The interface is designed using GUI components which are user-friendly.

Business Logic layer: is used for the retrieval of data obtained from user interfaces acquired form databases. Activities for calculating, managing and making decisions and communicating with the front-end application and back-end database are managed by business logic written using PHP language are appearing here.

Web Server layer: is to serve websites on the internet. It acts as a middleman between the server and client machines. It pulls content from the server on each user request and delivers it to the web. Apache Tomcat Web Server is used as a Web Server in the system.

Database layer: is used to store data forwarded by the business layer and retrieve that on demand to the business layer. MySQL database software is used as a back-end database of the system.

c. System Hardware Architecture

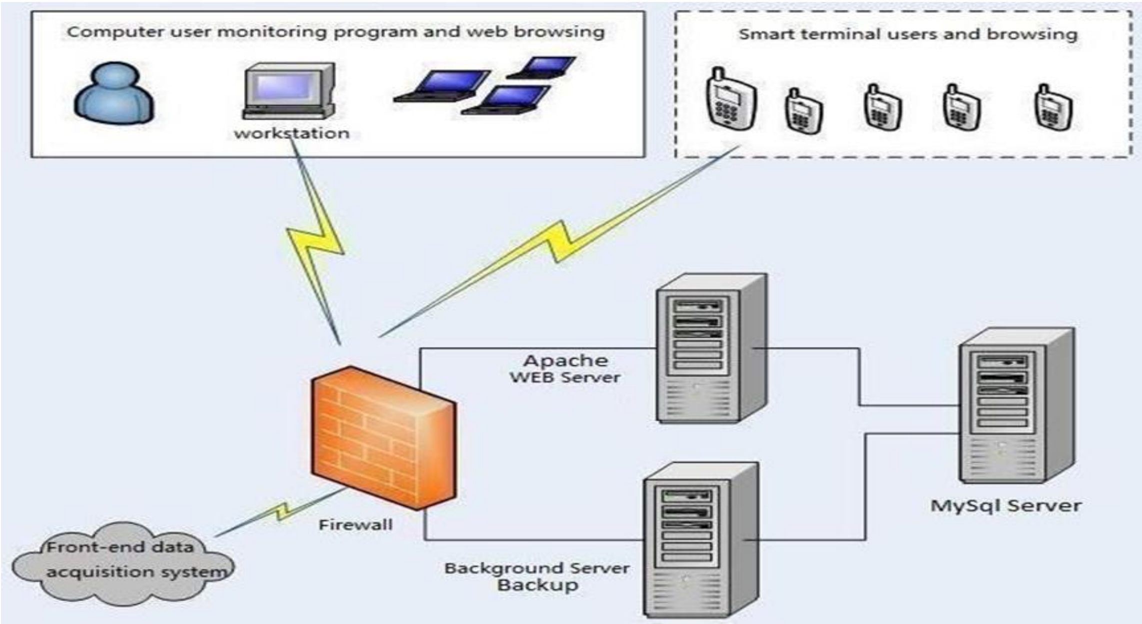


Figure 2.3: System Hardware Architecture

As shown in the figure above, the hardware architecture of the system consists of

a) Apache Tomcat Web Server

b) Alternate Backup Server

c) MYSQL database tool

d) Internet Connection

User devices such as Laptop, PC or Smartphones

2.1.2 Description of Programs

a. Context Flow Diagram (CFD)

The context diagram is used to establish the context and boundaries of the system to be modelled: which things are inside and outside of the system being modelled, and what is the relationship of the system with these external entities. A context diagram, sometimes called a level 0 data-flow diagram, is drawn in order to define and clarify the boundaries of the software system. It identifies the flows of information between the system and external entities. The entire software system is shown as a single process.

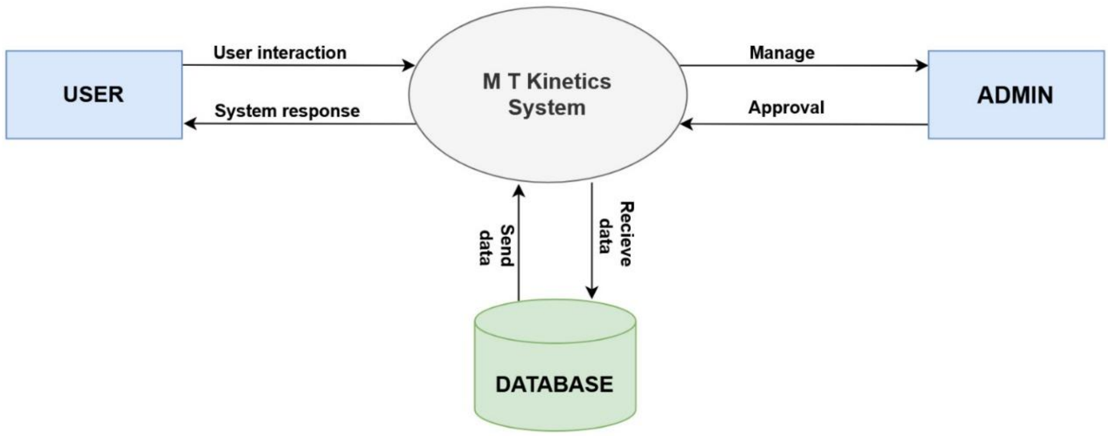


Figure 2.4: Context Flow Diagram

b. Data Flow Diagram (DFDs- Level 0, Level 1, Level 2)

Data Flow Diagram is a graphical representation of a system or a portion of the system. It consists of data flow, process, sources and sink and stores all the description through the use of easily understandable symbols. DFD is one of the most important modelling tools. It is used to model the system, components that interact with the system, use the data and information flows in the system. DFD shows the information moves through the and how it is modified by a series of transformations. It is a graphical technique that depicts information moves from input or output. DFD is also known as bubble chart or Data Flows Graphs. DFD may be used to represent the system at any level of abstraction.

- DFD is one of the most important modelling tools. It is used to model the system, components that interact with the system, uses the data and information flows in the system.
- DFD shows the information moves through the and how it is modified by a series of transformations. It is a graphical technique that depicts information moves from input or output.
- DFD is also knows as bubble chart or Data Flow Graphs. DFD may be used to represent the system at any level of abstraction. DFD's may partition into a level that represents increasing information flows and functional details.

b1. Data Flow Diagram (DFD's Level-0)

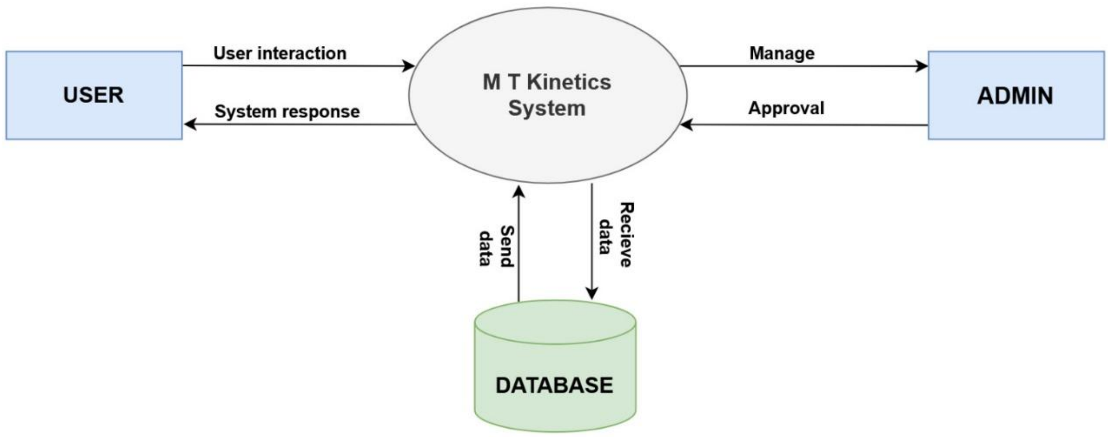


Figure 2.5: DFD level-0 Diagram

b2. Data Flow Diagram (DFD's Level-1)

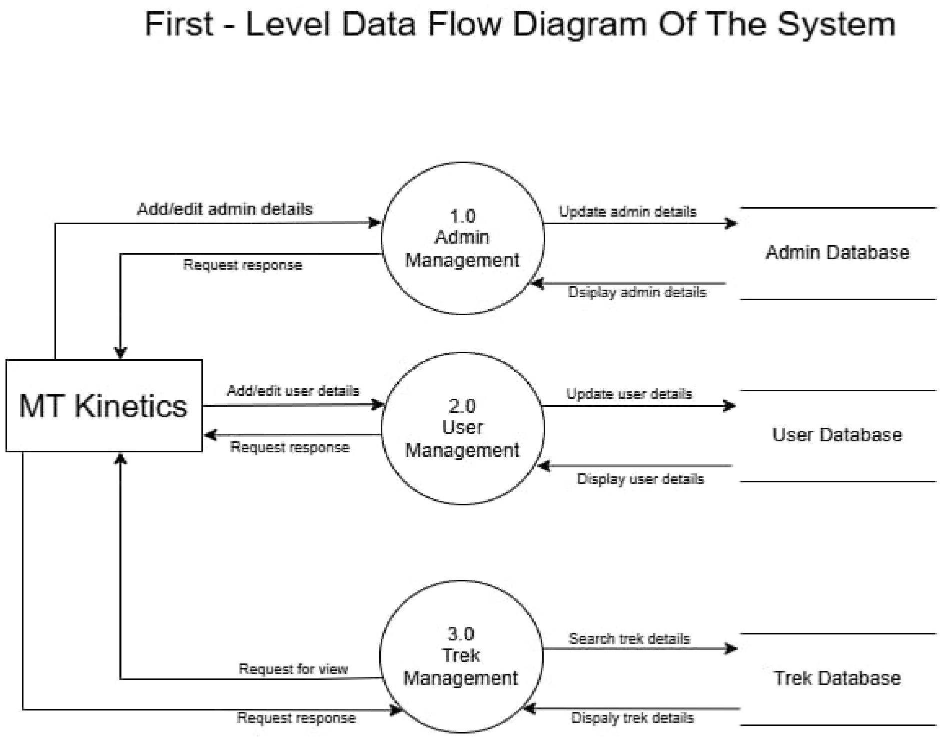


Figure 2.6: DFD level-1 Diagram

b3. Data Flow Diagram (DFD's Level-2)

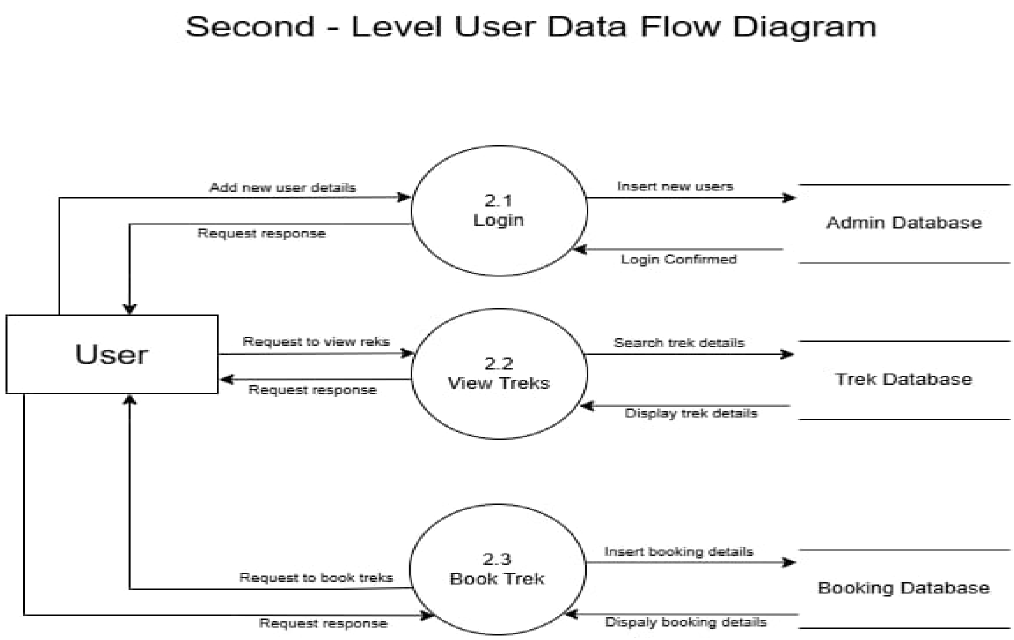


Figure 2.7: DFD level-2 Diagram

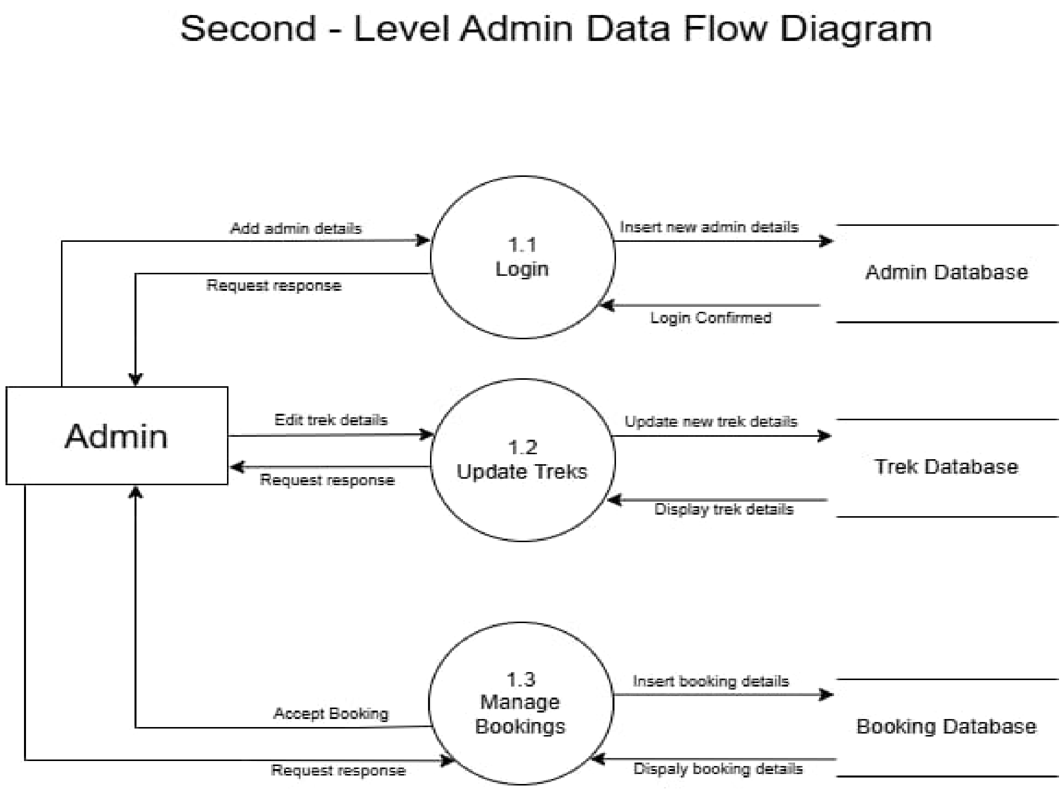


Figure 2.7: DFD level-2 Diagram

2.2 DATABASE DESIGN

2.2.1. Database Identification

- Primary key and foreign key are defined with same name.
- Make a separate table for each set of related attributes, and give each table a primary key.
- If an attribute depends on only part of a multi-valued key, remove it to a separate table.
- If attributes do not contribute to a description of the key, remove them to a separate table

2.2.2 Table Definition

A table definition is the blueprint that sets the structure. It defines:

- **Table Name:** Unique identifier for the data set.
- **Columns:** Data points (e.g., name, age) with data types (text, number).
- **Constraints:** Rules like primary keys (unique row identifiers) for data integrity.

User Table

Column Name	Data Type	Constraints
User_Id	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(150)	UNIQUE, NOT NULL
Password	VARCHAR(100)	NOT NULL

Admin Table

Column Name	**Data Type**	**Constraints**
Admin _Id	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NOT NULL
Password	VARCHAR(100)	NOT NULL

Trek Table

Column Name	**Data Type**	**Constraints**
Name	VARCHAR(100)	NOT NULL
Trek_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Location	VARCHAR(100)	NOT NULL
Price	INT	NOTNULL

Trek Table

Column Name	Data Type	Constraints
booking_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	NOT NULL
trek_id	INT	NOT NULL
num_persons	DATE	NOT NULL
trek_date	INT	NOT NULL

2.2.3 ER Diagram

E-R diagram or model is a type of structural diagram for use in database design. Entities are physical items or aggregations of data items that are important to the business we analyse or to the system; we intend to build. An entity represents an object defined within the information system about which you want to store information. Entities are named as singular nouns and are shown in rectangles in an ER-Diagram. Each entity is described by several attributes; individual instances of an entity will have different attribute values.

- Entity:

Entity is represented by a box within the ER Diagram. Entities are abstract concepts, each representing one or more instances of the concept in question. An entity might be considered.

- Relationship:

Relationships are represented by Diamonds. A relationship is a named collection or association between entities or used to relate to two or more entities with some common attributes or meaningful interaction between the objects.

- Attribute:

Attributes are represented by Oval. An attribute is a single data item related to a database object.

The database schema associates one or more attributes with each database entity.

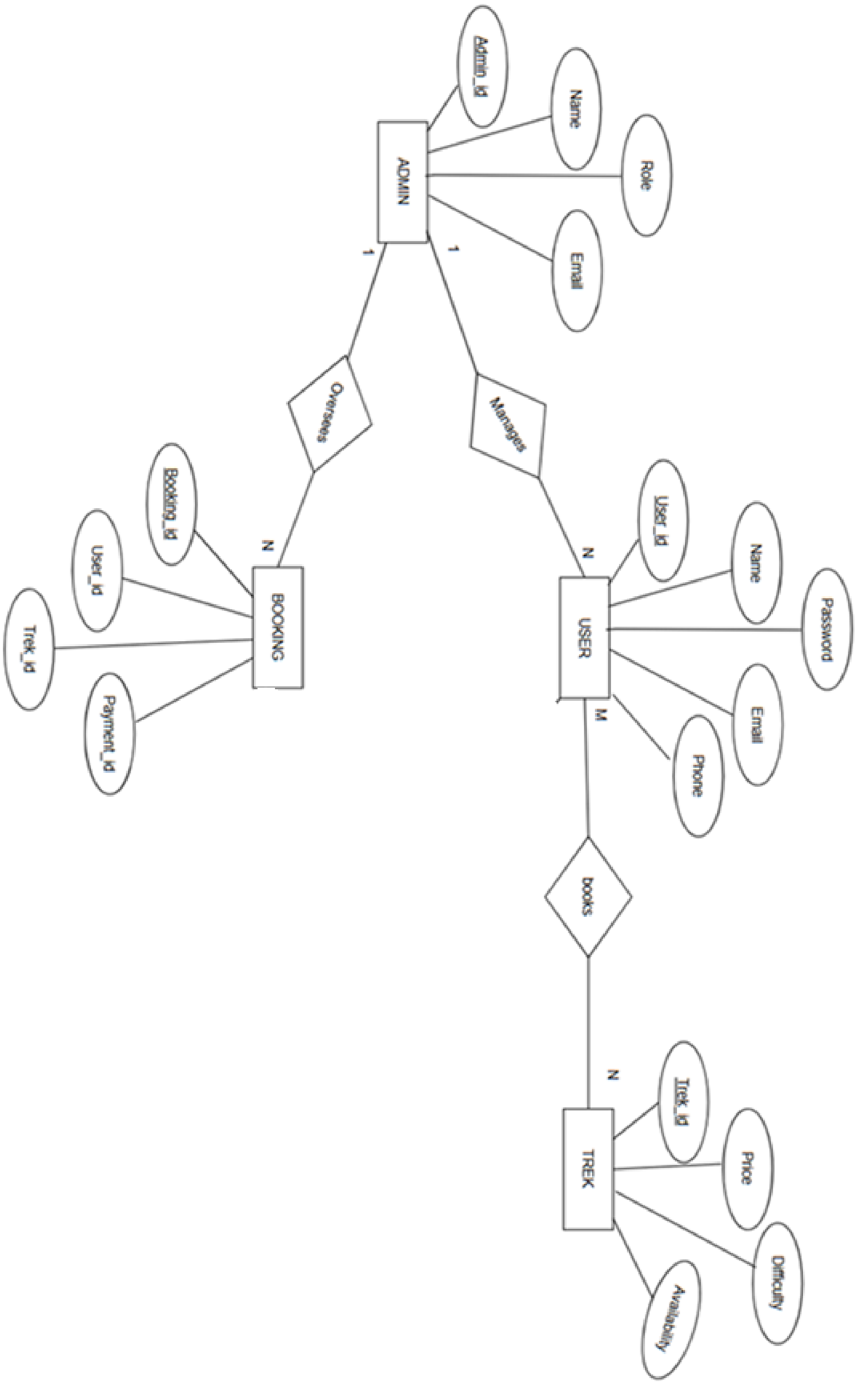


Figure 2.8: ER Diagram

2.3 DETAILED DESIGN

2.3.1 Modular Decomposition of the System

Another key component of the functional specification is a detailed design. For the detailed design you apply real-world technology constraints to the conceptual model, including implementation and performance considerations. Detailed design is a process of developing a fully defined product design from a clear set of requirements while creating deliverables and documentation appropriate for product manufacturing. The design of the system is perhaps a most critical factor affecting the quality of the software; it has a major impact on the later phases; particularly testing and maintenance. The output of this phase is the design document.

During this phase the design team uses both the requirement specification and the architectural specification provided by the previous phases to develop a detailed design of the system. This design will provide a detailed specification for each component, thoroughly describing interface and functions provided by each component. This detailed design will serve as the basis for the implementation phase.

2.3.1.1 Admin module

- Login to Admin Panel
- Manage Users
  - View/Delete Users
- Manage Treks
  - Add/Edit/Delete Trek Details
- Manage Bookings
  - View All Bookings
  - Confirm or Cancel Bookings

2.3.1.2 User module

- User Registration & Login
- View Trek Listings
- Book Trek
- View/Cancel Bookings
- Manage Profile

2.3.1.3 Trek module

- Trek Information
  - Name, Description, Location, Difficulty, Date
- Trek Availability
- Trek Gallery
- Trek Pricing
- Trek Booking Rules/Policies

2.3.2 Structure chart showing the hierarchy of the module

2.3.2.1 Structure chart of Admin

The admin structure chart represents the hierarchical organization of administrative functions within the MT Kinetics trekking management system. At the top is the Admin Dashboard, which branches into three main modules: Manage Users, Manage Booking, and Manage Trek. Each of these modules is further divided into specific sub-functions.

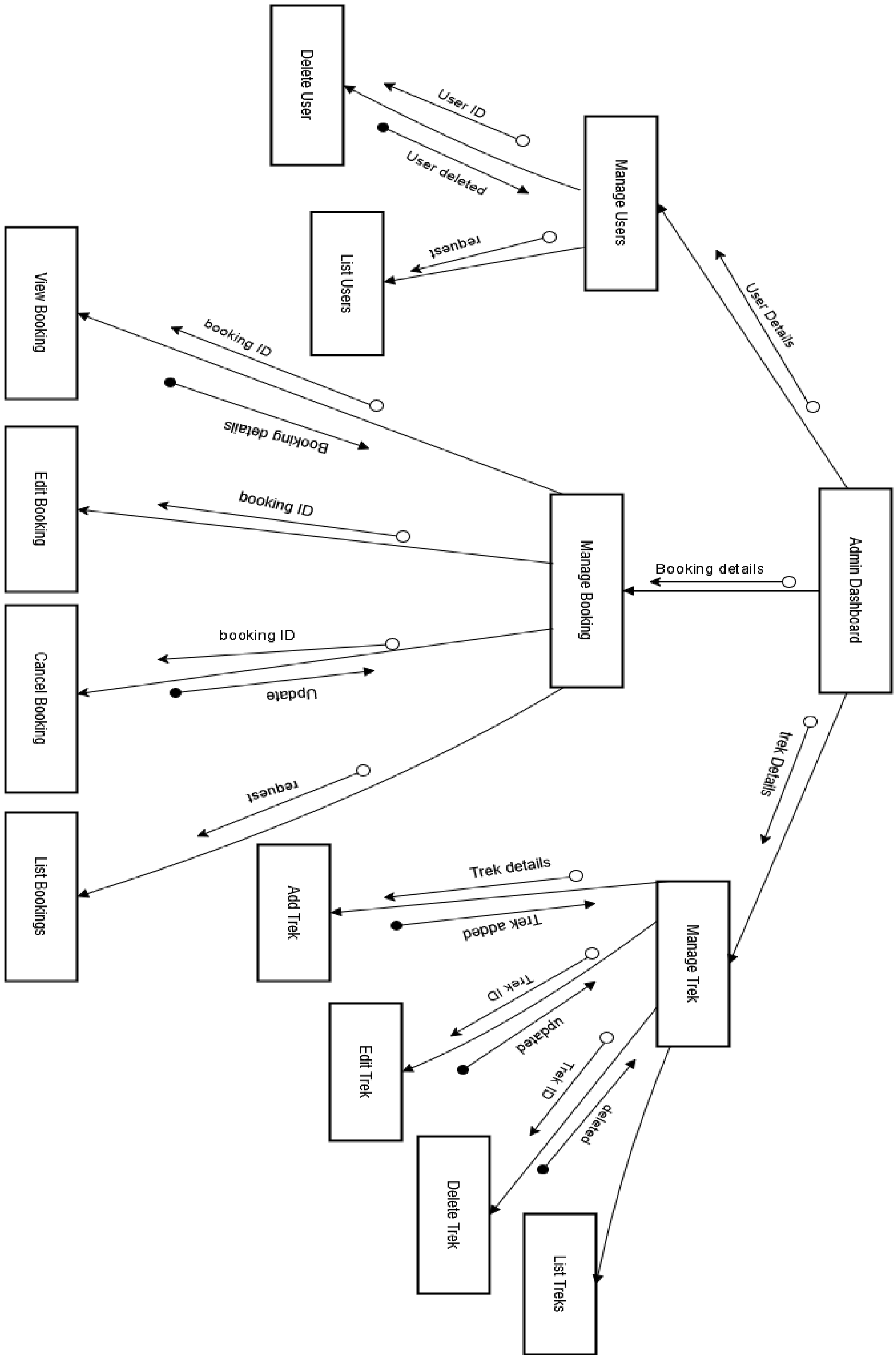


Figure 2.10: Structure Chart of Admin

2.3.2.2 Structure chart of User

This structure chart shows how a user interacts with the trekking system. After logging in or registering, the user accesses the dashboard to book treks, view trek details, cancel bookings, or manage their profile. Each action leads to related sub-modules, such as booking confirmation or cancellation confirmation. The system exchanges user data, trek details, and booking information to complete each operation.

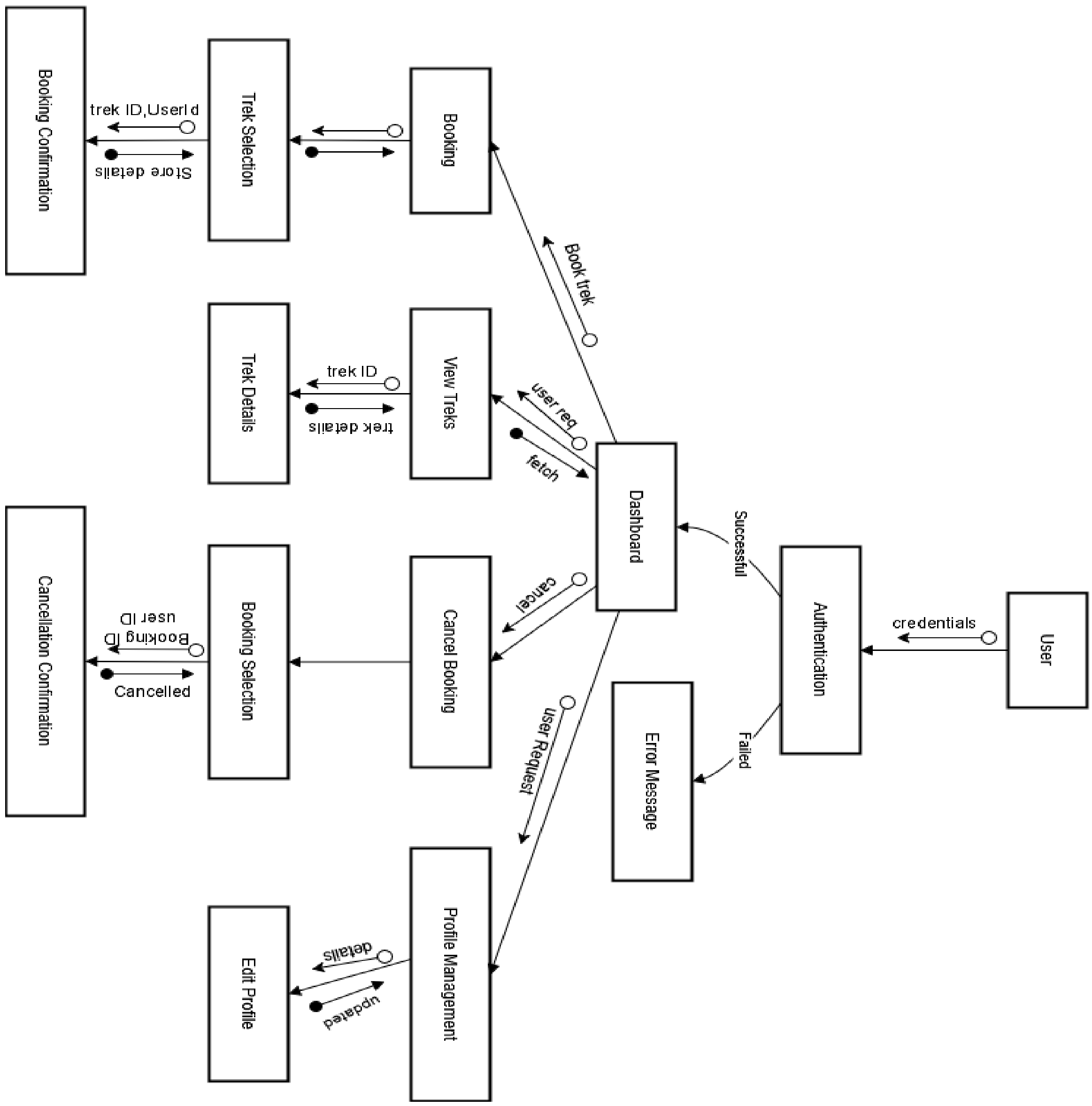


Figure 2.11: Structure Chart of Admin

2.3.3. Procedural Details of Components

Procedural Details (Flow Chart):

2.3.3.1 Admin login

Inputs: Admin username, password

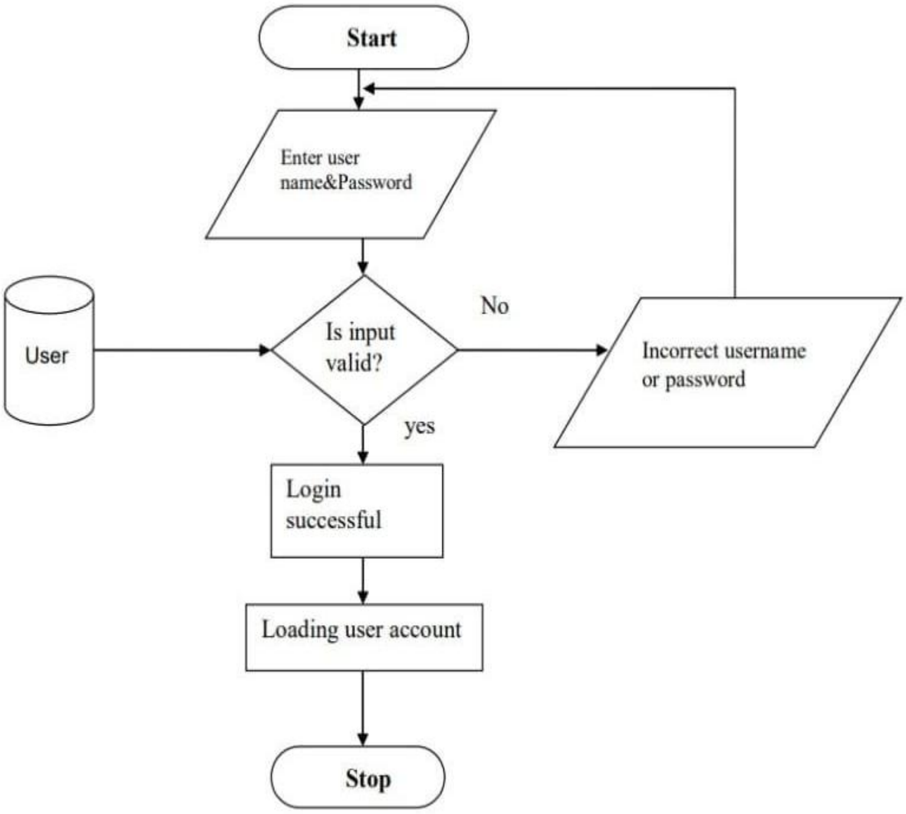


Figure 2.12: Flowchart Admin login

File Input/output: admin dashboard

Output: Entered Username and password will be checked for validity if it is valid Admin will be redirected to admin dashboard.

2.3.3.2 Manage Treks

Inputs: Trek Details

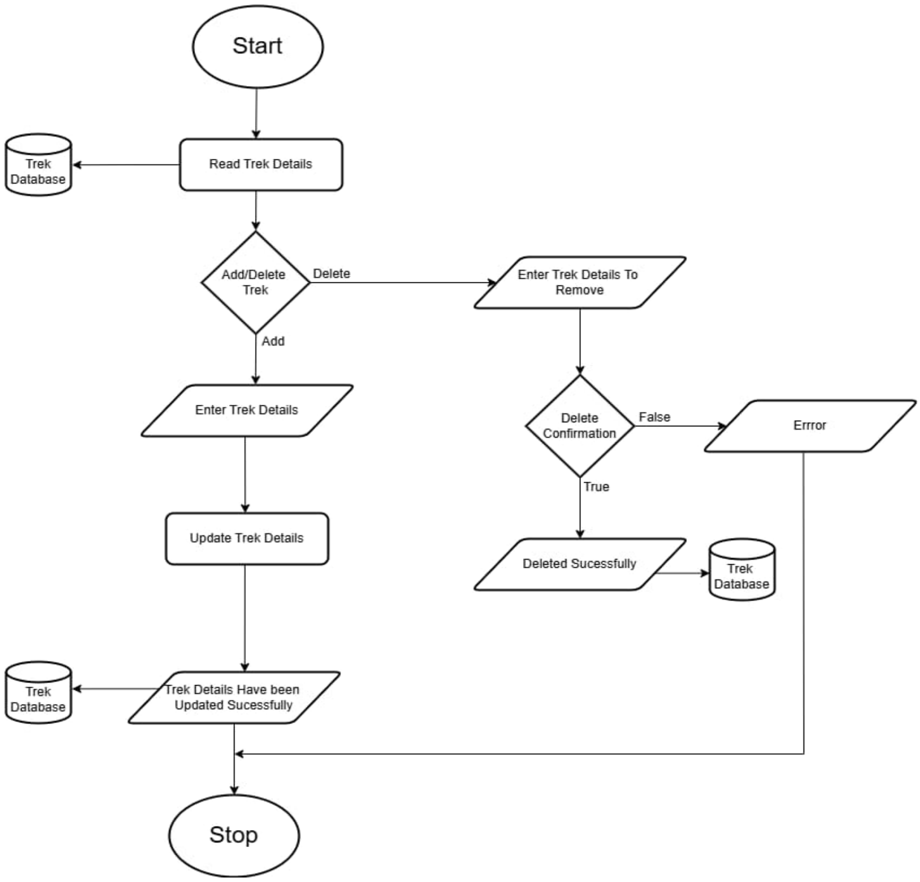


Figure 2.13 Admin Manages Trek details

File input/output interface: admin dashboard

Output: Trek details are modified or updated

2.3.3.3 Manage User Bookings

Input: Booking ID, user details

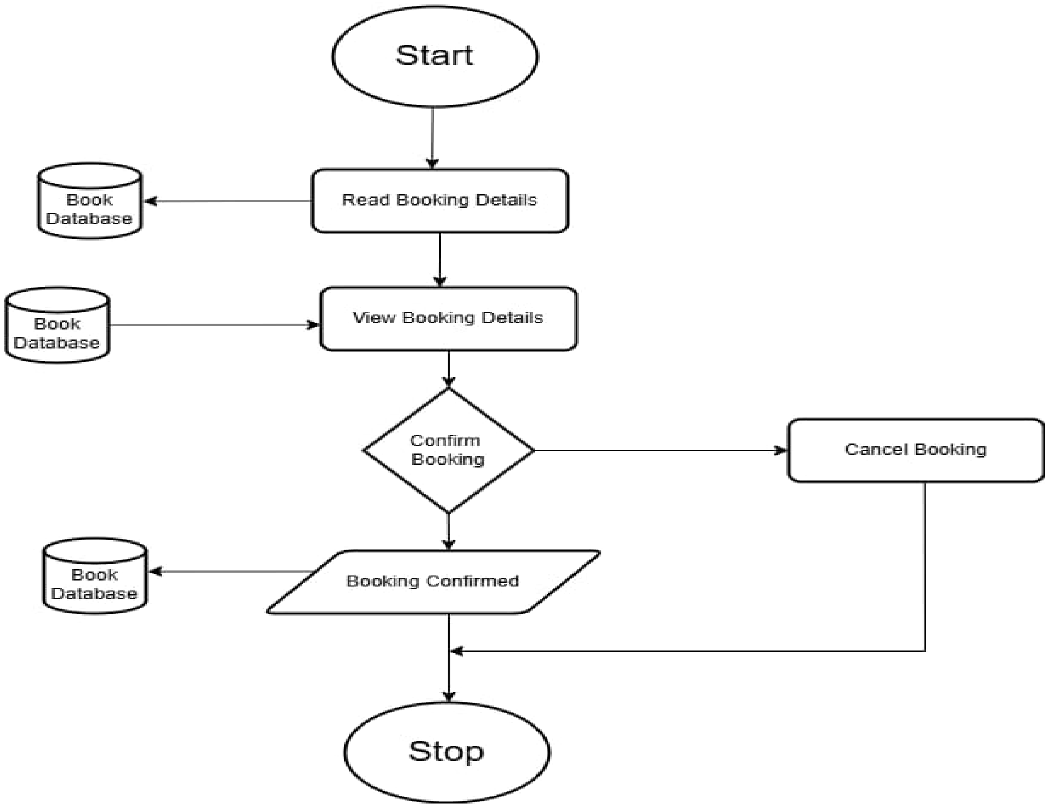


Figure 2.14 Admin Manages Booking

File input/output interface: Admin dashboard

Output: bookings are accepted or cancel according to admin response

2.3.3.5 User Login

Input: username, password, email

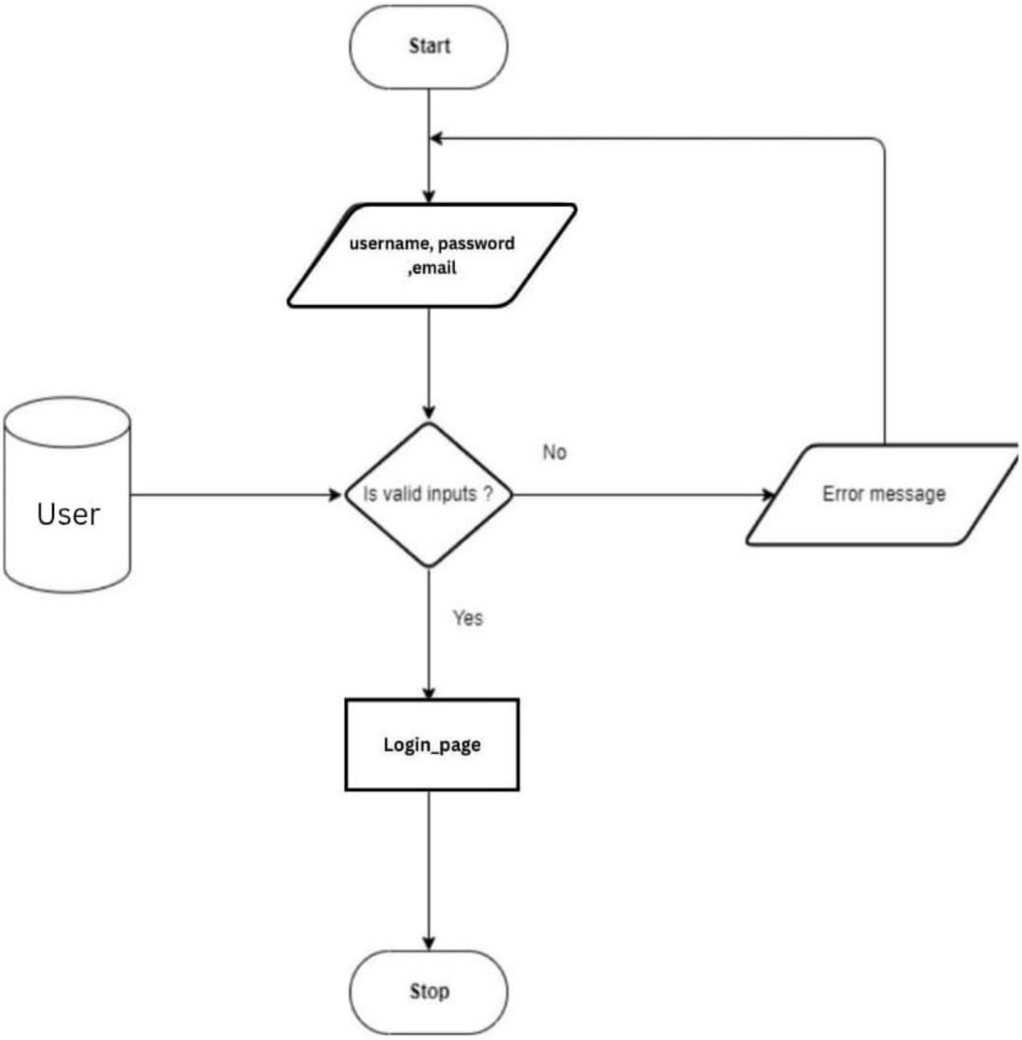


Figure 2.15 User Login

File input/output interface: Login successful

2.3.3.6 User View Trek

Input: Click View trek

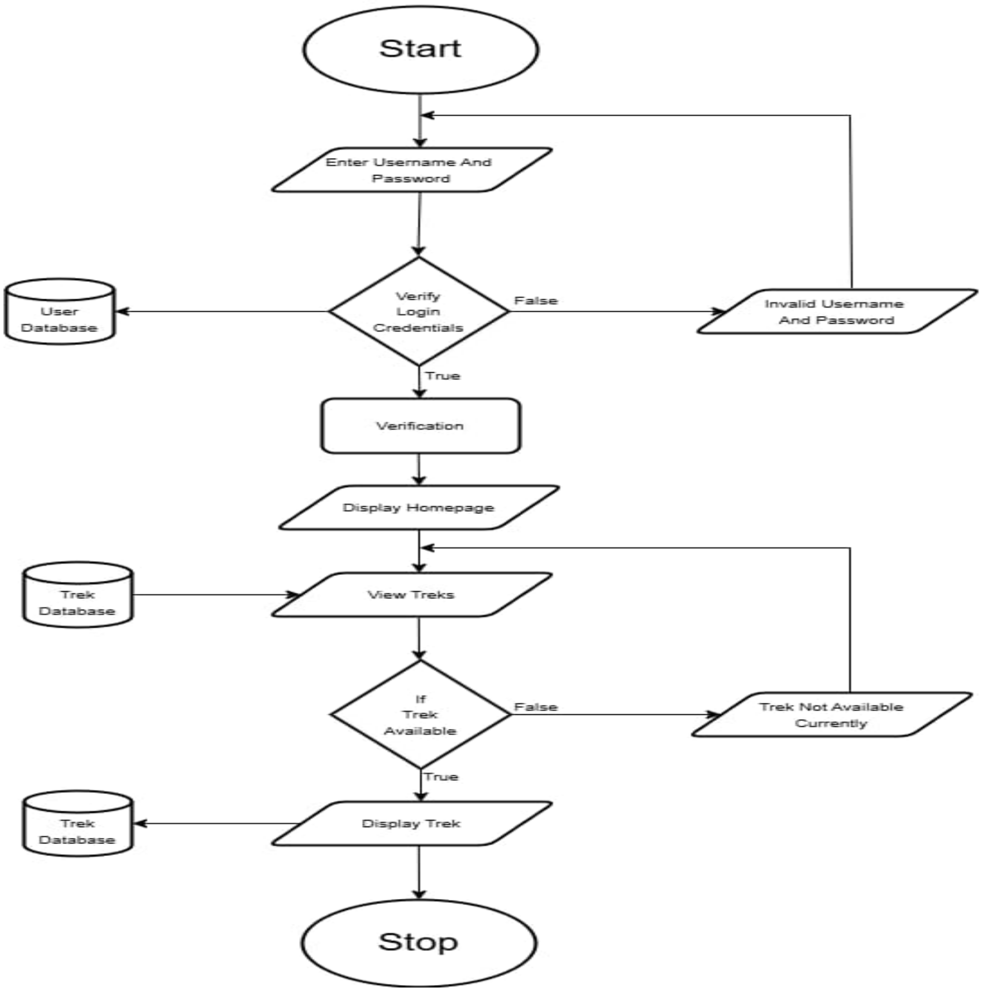


Figure 2.16 View Trek

File input/output interface: user view page

Output: Trek details are displayed

2.3.3.6 User Book Trek

Input: Click Book trek on selected Trek

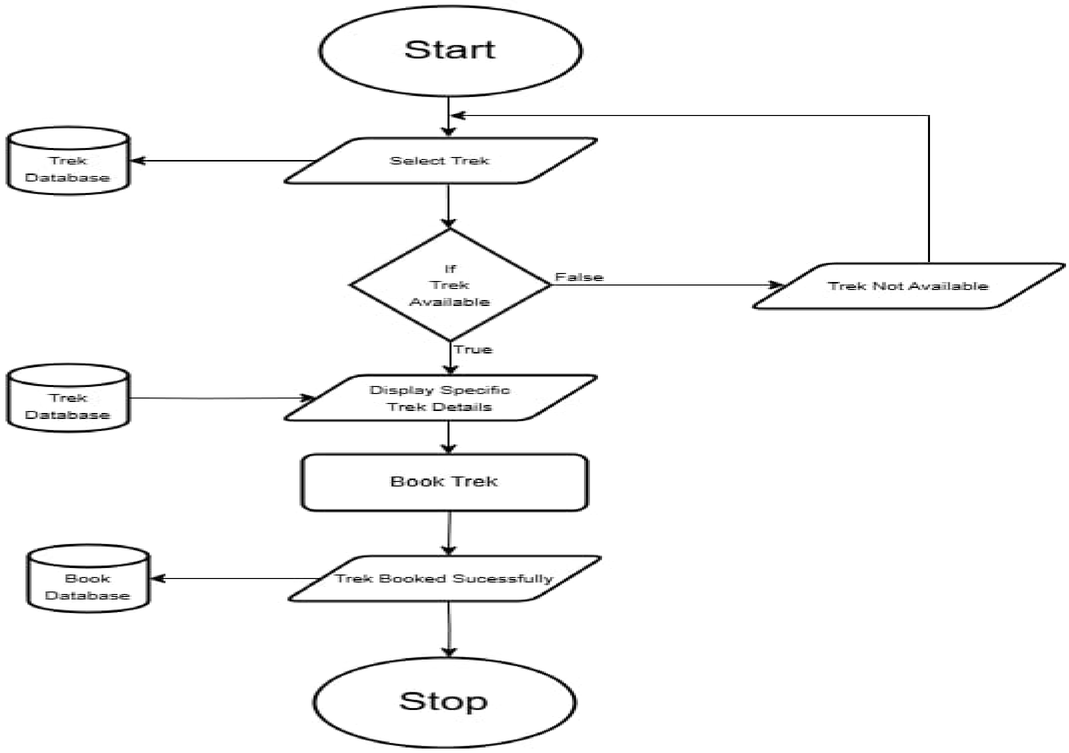


Figure 2.17 Book Trek

File input/output interface: User Booking Page

Output: Trek Booked Successfully

Chapter – 3

ANALYSIS AND INTERPRETATION

3.1 PROGRAM CODE LISTING

3.1.1 Database Connection

```
<?php
// php/db.php

$host = 'localhost';
$db   = 'mt_kinetics';
$user = 'root'; // use your MySQL username
$pass = '';     // use your MySQL password
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

$options = [
            PDO::ATTR_ERRMODE                          =>
PDO::ERRMODE_EXCEPTION,
                 PDO::ATTR_DEFAULT_FETCH_MODE          =>
PDO::FETCH_ASSOC,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
?>
```

3.1.2 Authorization/Authentication

Admin login

```
<?php
require 'db.php';
session_start();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'];
    $pass = $_POST['password'];

    $stmt = $pdo->prepare("SELECT * FROM admins WHERE email = ?");
    $stmt->execute([$email]);
    $admin = $stmt->fetch();

    if ($admin && password_verify($pass, $admin['password'])) {
        $_SESSION['admin_id'] = $admin['admin_id'];
        $_SESSION['role'] = 'admin';
        header("Location: admin/index.html");
    } else {
        echo "Invalid credentials.";
    }
}
?>
```

User Registration

```
<?php
require 'db.php';
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name     = $_POST['name'];
    $email    = $_POST['email'];
     $phone    = $_POST['phone'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
     $stmt = $pdo->prepare("INSERT INTO users (name, email, password,phone_number)
VALUES (?, ?, ?,?)");
    if ($stmt->execute([$name, $email, $password,$phone])) {
        header("Location: ../login.html");
        exit();
    } else {
        echo "Registration failed.";
    }
}
?>
```

User Login

```php
<?php
require 'db.php';
session_start();
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $email    = $_POST['email'];
    $password = $_POST['password'];

    $loggedIn = false;

    // Try admin login first (often preferred if admin has higher privileges)
    $stmt = $pdo->prepare("SELECT * FROM admins WHERE email = ?");
    $stmt->execute([$email]);
    $admin = $stmt->fetch();

    if ($admin && password_verify($password, $admin['password'])) {
        $_SESSION['admin_id'] = $admin['admin_id'];
        $_SESSION['role'] = 'admin';
        header("Location: ../admin/index.html");
        $loggedIn = true;
        exit(); // Exit after a successful admin login
    }

    // Only try user login if admin login failed
    if (!$loggedIn) {
        $stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
        $stmt->execute([$email]);
        $user = $stmt->fetch();

        if ($user && password_verify($password, $user['password'])) {
            $_SESSION['user_id'] = $user['user_id'];
            $_SESSION['role'] = 'user';
            header("Location: ../user/index.html");
            $loggedIn = true;
            exit(); // Exit after a successful user login
        }
    }

    // If neither user nor admin login succeeds
    if (!$loggedIn) {
        header("Location: ../extra/error.html");
        exit();
    }
}
?>
```

User Foget Password

```php
<?php
require_once "db.php"; // This sets up $pdo

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $email = trim($_POST["email"]);
   $password = trim($_POST["password"]);

   if (empty($email) || empty($password)) {
      echo "<script>alert('All fields are required.'); window.history.back();</script>";
      exit;
   }

   // Check if email exists
   $stmt = $pdo->prepare("SELECT user_id FROM users WHERE email = ?");
   $stmt->execute([$email]);

   if ($stmt->rowCount() > 0) {
      // Hash password
      $hashed_password = password_hash($password, PASSWORD_DEFAULT);

      // Update password
      $update = $pdo->prepare("UPDATE users SET password = ? WHERE email = ?");
      if ($update->execute([$hashed_password, $email])) {
            echo "<script>alert('Password reset successfully.');
window.location.href='../login.html';</script>";
      } else {
         echo "<script>alert('Error updating password.'); window.history.back();</script>";
      }
   } else {
         echo "<script>alert('No user found with that email.');
window.history.back();</script>";
   }
} else {
   header("Location: login.html");
   exit;
}
?>
```

3.1.3 Admin Operations

Add Trek

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db = "mt_kinetics";

$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}

$trek_code = "TREK" . date("YmdHis");

// Handle image upload
$target_dir = "uploads/";
if (!file_exists($target_dir)) {
   mkdir($target_dir, 0777, true);
}
$image_name = basename($_FILES["image"]["name"]);
$image_path = $target_dir . $trek_code . "_" . $image_name;
move_uploaded_file($_FILES["image"]["tmp_name"], $image_path);

// Get form data
$name = $_POST['name'];
$location = $_POST['location'];
$date = $_POST['date'];
$price = $_POST['price'];
$description = $_POST['description'];

// Insert into database
$sql = "INSERT INTO treks (trek_code, name, location, date, price,
description, image_path)
        VALUES (?, ?, ?, ?, ?, ?, ?)";
$stmt = $conn->prepare($sql);
$stmt->bind_param("ssssiss", $trek_code, $name, $location, $date, $price,
$description, $image_path);
$stmt->execute();
if ($stmt->affected_rows > 0) {
            echo "Trek added successfully with code:
<strong>$trek_code</strong><br>";
   echo '<a href="../admin/index.html">Go back to Admin Dashboard</a>';
```

```
} else {
    echo "Failed to add trek.<br>";
    echo '<a href="../admin/index.html">Go back to Admin Dashboard</a>';
}

$stmt->close();
$conn->close();
?>
```

Delete User

```
<?php
require 'db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST' &&
isset($_POST['user_id'])) {
    $user_id = $_POST['user_id'];

    // Delete bookings first
     $pdo->prepare("DELETE FROM bookings WHERE user_id = ?")-
>execute([$user_id]);

    // Then delete user
      $pdo->prepare("DELETE FROM users WHERE user_id = ?")-
>execute([$user_id]);
}

header('Location: ../admin/users/user.html');
exit();
?>
```

Get Bookings

```
<?php
session_start();
require 'db.php';

if (!isset($_SESSION['admin_id'])) {
    echo json_encode([]);
    exit();
}

$stmt = $pdo->query("SELECT b.*, u.email FROM bookings b JOIN users
u ON b.user_id = u.user_id ORDER BY booking_date DESC");
$bookings = $stmt->fetchAll();
```

```
echo json_encode($bookings);
```

Get Trek details

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db = "mt_kinetics";

$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}

$result = $conn->query("SELECT trek_code, name, location, date, price,
description, image_path FROM treks ");
$treks = [];

while ($row = $result->fetch_assoc()) {
   $treks[] = $row;
}

header('Content-Type: application/json');
echo json_encode($treks);
$conn->close();
?>
```

Get User Details

```
<?php
require 'db.php';
$stmt = $pdo->query("SELECT user_id, name, email, phone_number FROM
users");
$users = $stmt->fetchAll();
header('Content-Type: application/json');
echo json_encode($users);
?>
```

Approve Booking

```
<?php
session_start();
require '../php/db.php';

if (!isset($_SESSION['admin_id'])) {
   header("Location: ../index.html");
   exit();
}

if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['booking_id'])) {
    $stmt = $pdo->prepare("UPDATE bookings SET status = 'Approved' WHERE booking_id = ?");
   $stmt->execute([$_POST['booking_id']]);
}

header("Location: view_booking.html");
exit();
?>
```

Cancel Booking

```
<?php
session_start();
require '../php/db.php';

if (!isset($_SESSION['admin_id'])) {
   header("Location: ../index.html");
   exit();
}

if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['booking_id'])) {
    $stmt = $pdo->prepare("UPDATE bookings SET status = 'Cancelled' WHERE booking_id = ?");
   $stmt->execute([$_POST['booking_id']]);
}

header("Location: view_booking.html");
exit();
?>
```

3.1.4 User Operations

User Book trek

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db = "mt_kinetics";
require 'db.php'; // correct path if book_trek.php is in php folder and db.php
also in same folder

$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
   $trek_name = $_POST['trek_name'] ?? '';
   $trek_code = $_POST['trek_code'] ?? '';
   $email = $_POST['email'] ?? '';
   $num_participants = $_POST['num_participants'] ?? 1;
   $comments = $_POST['comments'] ?? '';

   // Optional: Validate email exists in users table
   $stmt = $pdo->prepare("SELECT user_id FROM users WHERE email =
?");
   $stmt->execute([$email]);
   $user = $stmt->fetch();

   if (!$user) {
      echo "User not found. Please register or check your email.";
      exit();
   }

    $stmt = $pdo->prepare("INSERT INTO bookings (user_id, trek_name,
trek_code, num_participants, comments, status, booking_date)
                     VALUES (?, ?, ?, ?, ?, 'Pending', NOW())");

         if ($stmt->execute([$user['user_id'], $trek_name, $trek_code,
$num_participants, $comments])) {
      header("Location: ../user/confirmation.html");
      exit();
   } else {
      echo "Failed to book trek. Please try again.";
   }
```

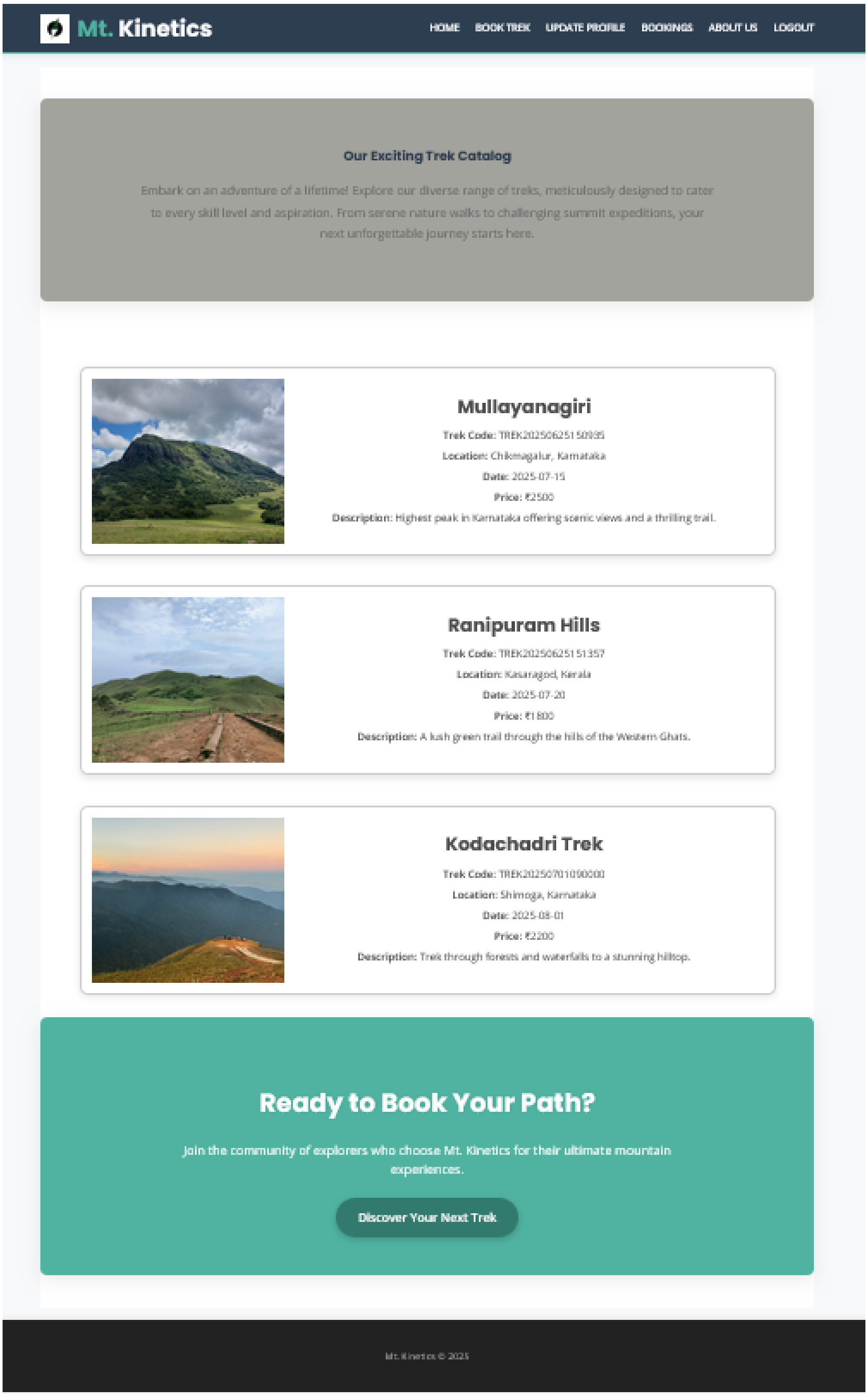
```
} else {
    echo "Invalid request.";
}
?>
```

Update Profile

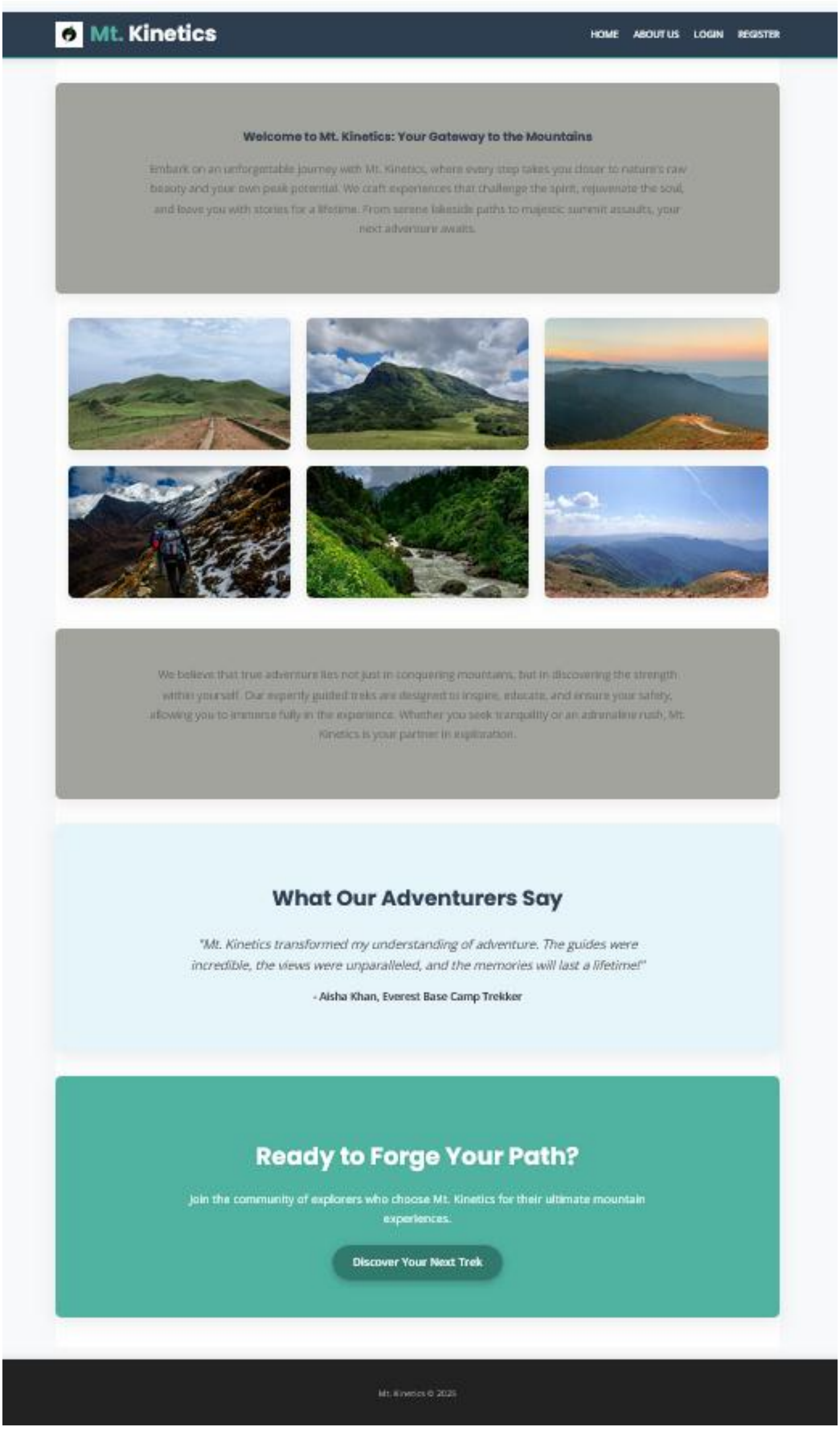
```
<?php
session_start();
require 'db.php';

if (isset($_SESSION['user_id'])) {
    // Hash the password before saving
    $hashedPassword = password_hash($_POST['password'], PASSWORD_DEFAULT);

    // Prepare and execute the update query
    $stmt = $pdo->prepare("UPDATE users SET name = ?, email = ?, password = ?, phone_number = ? WHERE user_id = ?");
    $stmt->execute([
        $_POST['name'],
        $_POST['email'],
        $hashedPassword,
        $_POST['phone'],
        $_SESSION['user_id']
    ]);

    // Force logout so user can log in with new password
    header("Location: ../logout.php");
    exit();
}
?>
```

3.2. User Interface

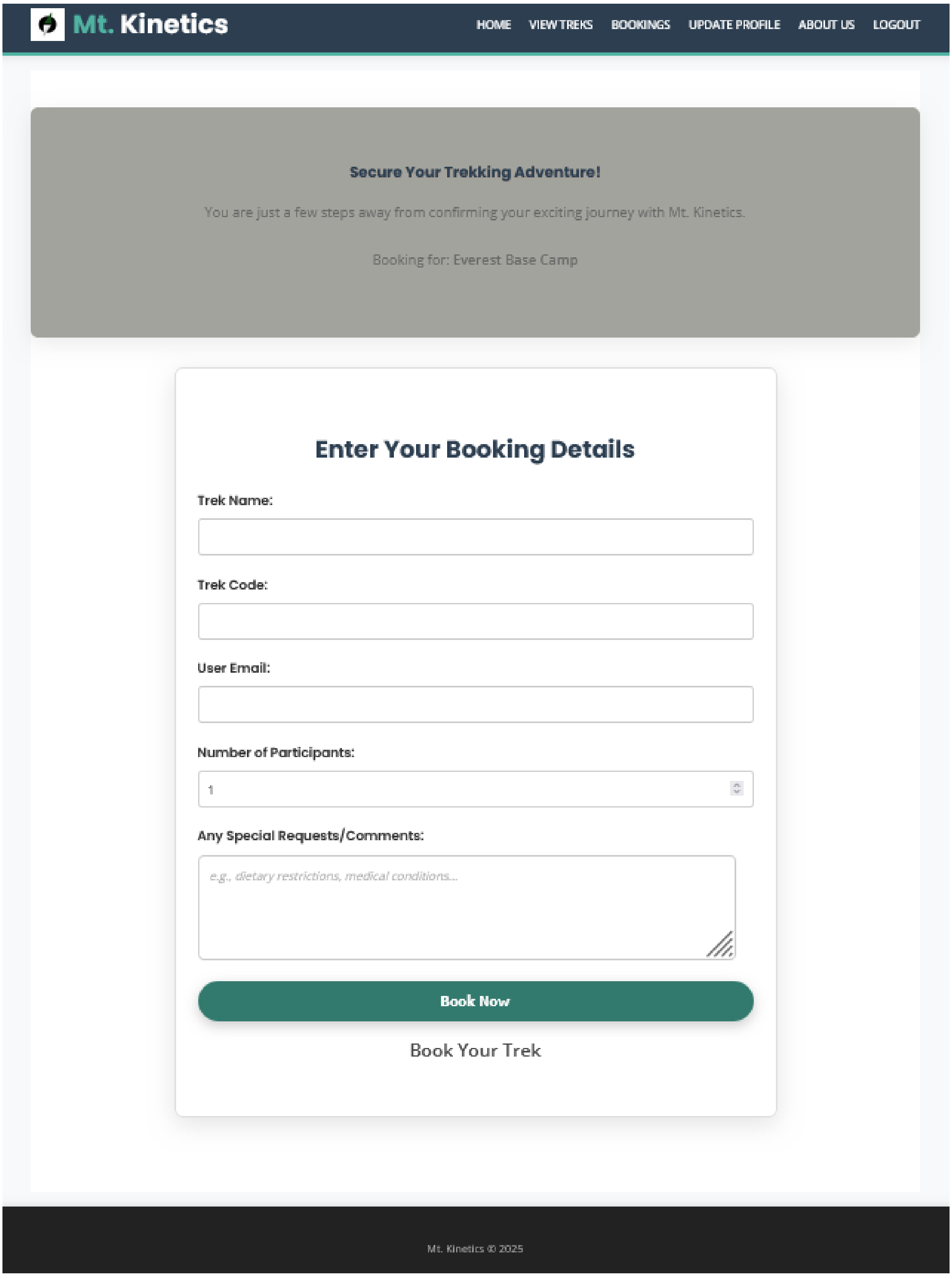
View_Trek page:



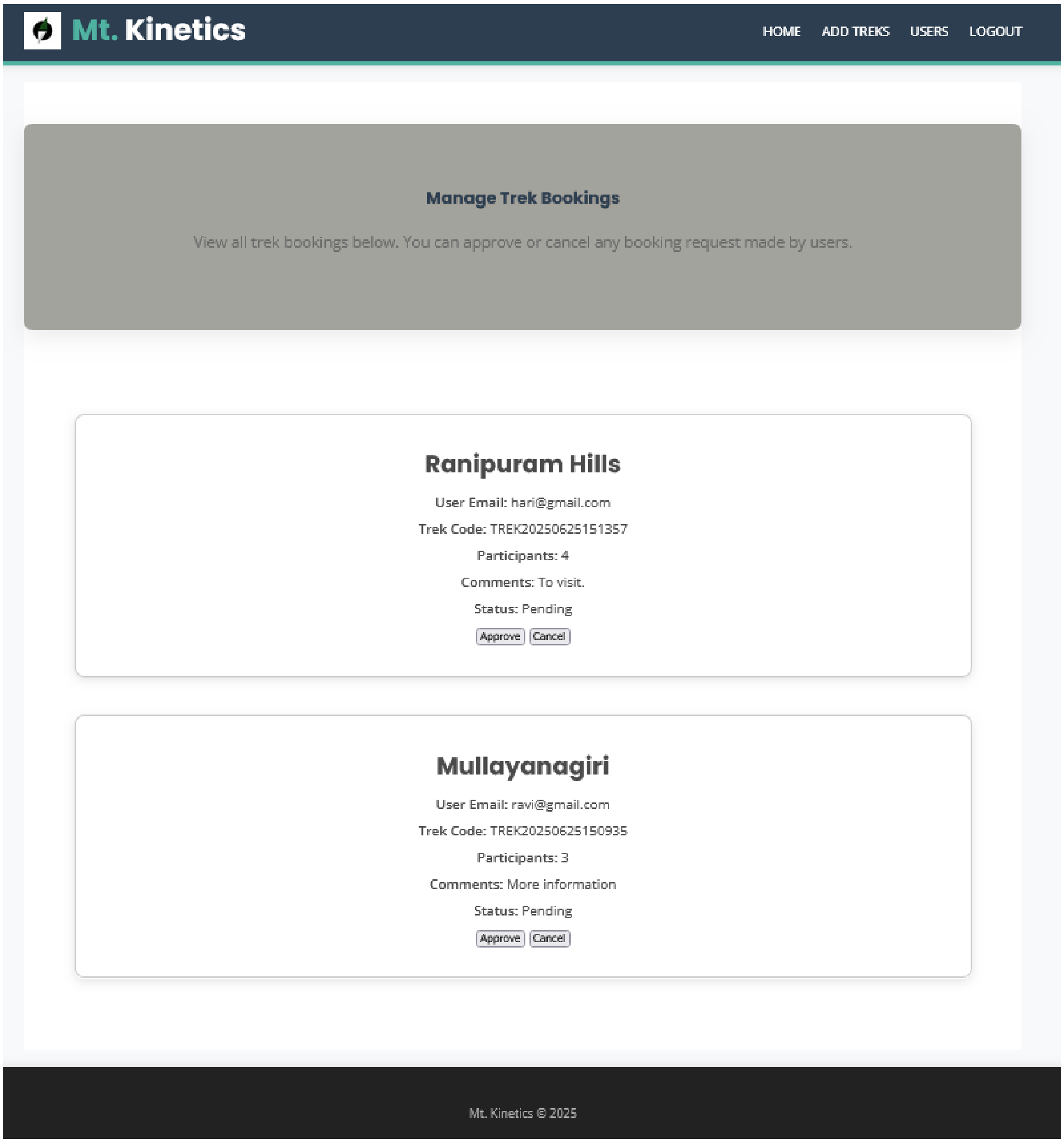
Home page:


Mt. Kinetics
HOME
ABOUT US
LOGIN
REGISTER
Welcome to Mt. Kinetics: Your Gateway to the Mountains
Embark on an unforgettable journey with Mt. Kinetics, where every step takes you closer to nature's raw beauty and your own peak potential. We craft experiences that challenge the spirit, rejuvenate the soul, and leave you with stories for a lifetime. From serene lakeside paths to majestic summit assaults, your next adventure awaits.
We believe that true adventure lies not just in conquering mountains, but in discovering the strength within yourself. Our expertly guided treks are designed to inspire, educate, and ensure your safety, allowing you to immerse fully in the experience. Whether you seek tranquility or an adrenaline rush, Mt. Kinetics is your partner in exploration.
What Our Adventurers Say
"Mt. Kinetics transformed my understanding of adventure. The guides were incredible, the views were unparalleled, and the memories will last a lifetime!"
- Aisha Khan, Everest Base Camp Trekker
Ready to Forge Your Path?
Join the community of explorers who choose Mt. Kinetics for their ultimate mountain experiences.
Discover Your Next Trek

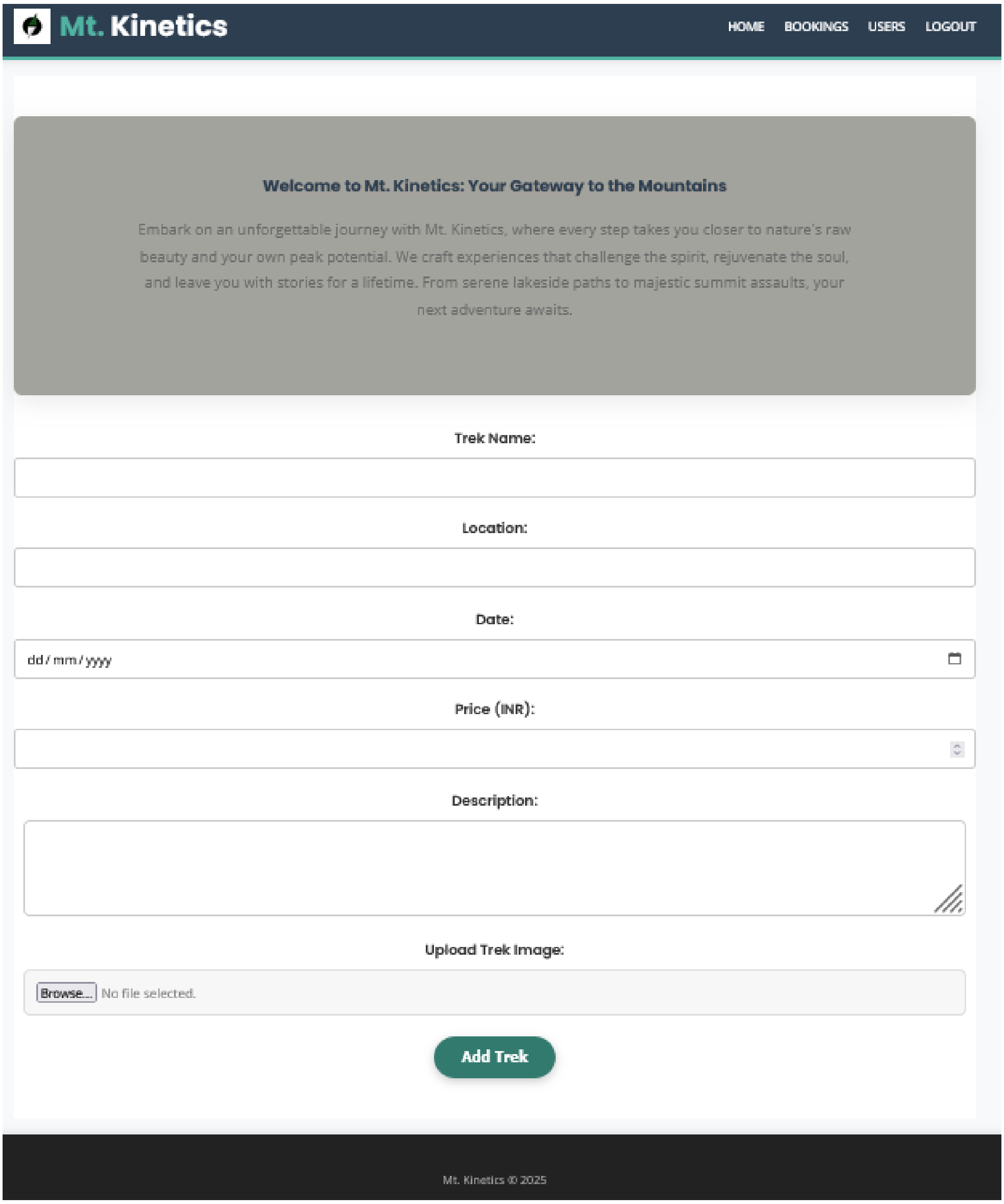
Booking page:



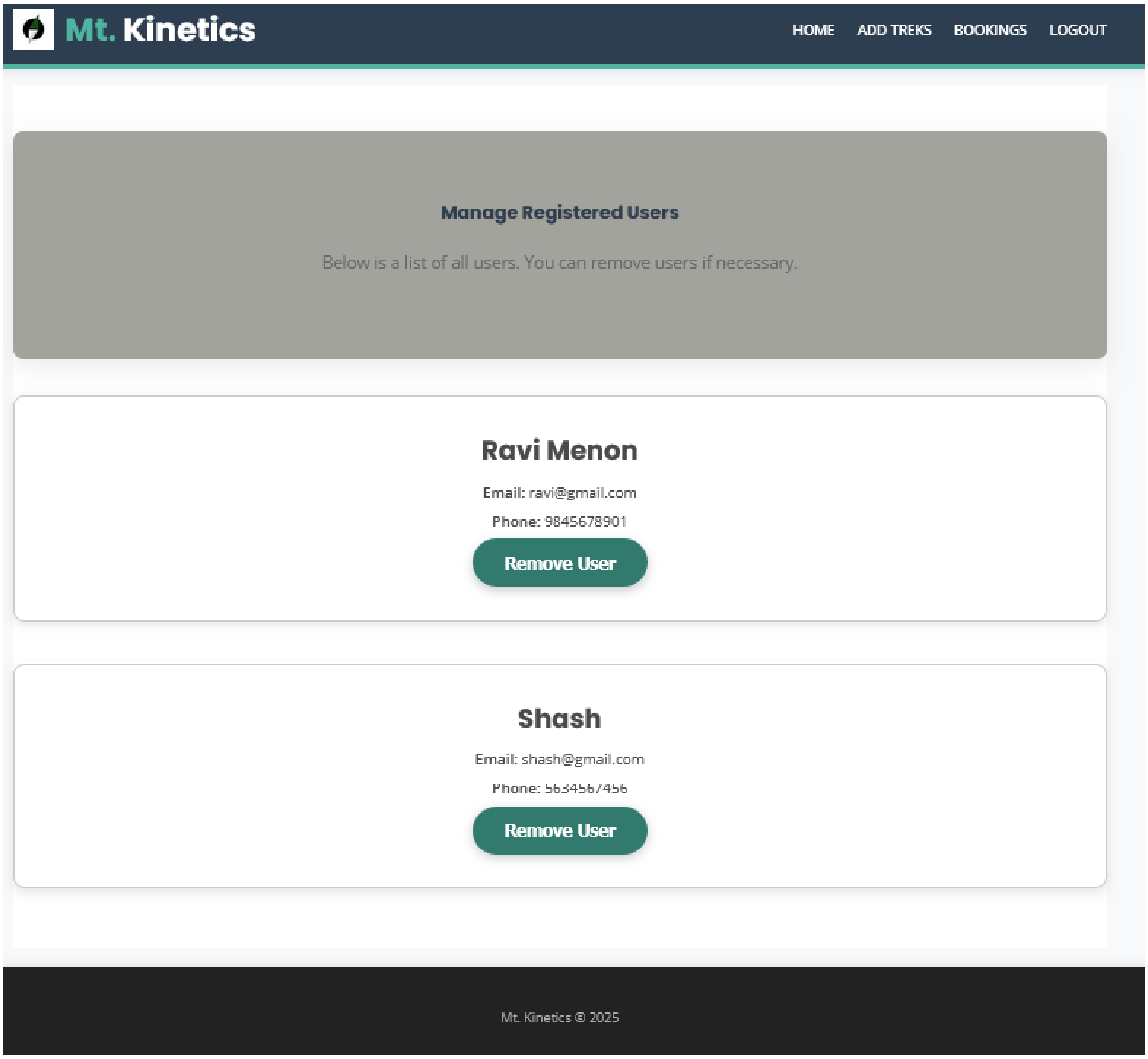
Booking Confirmation Page:



Add Trek Page:



User Management Page:



Chapter – 4

CONCLUSION

4.1 CONCLUSION

The Tour Package Booking System for MT Kinetics successfully meets the goals set out during project planning. It bridges the gap between trekkers and tour operators by providing a digital platform for discovering, booking, and managing trekking adventures. The system enhances user satisfaction through a well-structured interface and contributes to operational efficiency for MT Kinetics. This solution can be expanded in the future with features like real-time chat, GPS-based route maps, or third-party payment gateway integrations.

4.2 MAJOR FINDINGS

The development of the "Online Tour Package Booking System for MT Kinetics" led to several key findings:

1. User Convenience: The web application provides a simple and intuitive interface for customers to explore and book trekking packages around Mangalore and Bangalore.
2. Efficient Admin Management: The admin panel simplifies the management of packages, bookings, and customer data.
3. Secure Booking System: With PHP and MySQL integration, user credentials and booking data are handled securely.
4. Responsive Design: The frontend, built with HTML, CSS, and JavaScript, ensures usability across various devices.
5. Automated Workflow: The system automates tasks like booking confirmation, user registration, and data management, reducing manual intervention.

4.3 LIMITATIONS

- The system currently lacks a payment gateway integration; bookings are assumed to be confirmed offline.
- Real-time seat availability is not synchronized; it depends on admin updates.
- No mobile app version exists — only a web application is available.
- Does not currently support multilingual functionality.
- Limited analytics and reporting tools for the admin panel.

4.4 SUGGESTIONS / RECOMMENDATIONS

- Integrate online payment options like Razorpay, Paytm, or UPI.
- Implement real-time booking status updates.
- Add mobile app support for Android and iOS.
- Introduce user reviews and ratings for transparency.
- Incorporate email/SMS notifications for bookings and updates.
- Enhance admin dashboard with data visualization tools.

4.5 LEARNING OUTCOMES

- Gained practical experience in web development using HTML, CSS, JavaScript, PHP, and MySQL.
- Understood system development life cycle (SDLC) including planning, design, development, and testing.
- Developed database design skills through ER diagrams and normalized schema.
- Learned how to create user-centric UIs and manage session-based authentication.
- Enhanced debugging, problem-solving, and documentation skills.

4.7 ABBREVIATIONS AND ACRONYMS

- DFD: Data Flow Diagram
- GUI: Graphical User Interface
- CFD: Context Flow Diagram
- ER: Entity-Relationship Diagram
- PK: Primary Key
- FK: Foreign Key
- SRS: System Requirement Specification
- DBMS: Database Management System
- HTML: HyperText Markup Language
- CSS: Cascading Style Sheets
- PHP: Hypertext Preprocessor
- SQL: Structured Query Language
- API: Application Programming Interface

4.8 REFERENCES

- Duckett, J. (2011). *HTML and CSS: Design and Build Websites.* John Wiley & Sons. https://wtf.tw/ref/duckett.pdf
- Welling, L., & Thomson, L. (2009). *PHP and MySQL Web Development* (4th ed.). Addison-Wesley. https://repository.unikom.ac.id/
- Ullman, L. (2011). *PHP for the Web: Visual QuickStart Guide* (4th ed.). Peachpit Press. https://larryullman.com/books/php-for-the-web-visual-quickstart-guide-4th-edition/
- MDN Web Docs. (n.d.). *HTML: HyperText Markup Language.* https://developer.mozilla.org/en-US/docs/Web/HTML
- W3Schools. (n.d.). *PHP Tutorial.* https://www.w3schools.com/php/default.asp